

บทที่ 9: การจัดการข้อผิดพลาด (Exception Handling)

บรรยายโดย ผศ.ดร.ธราวิเชษฐ์ ธิติจรูญโรจน์

คณะเทคโนโลยีสารสนเทศ

สถาบันเทคโนโลยีพระจอมเกล้าเจ้าคุณทหารลาดกระบัง

หัวข้อ

- ข้อผิดพลาดคืออะไร
- การสร้างคลาสประเภทข้อผิดพลาดขึ้นมาใหม่
- วิธีการจัดการข้อผิดพลาด
- หลักการจัดการกับข้อผิดพลาด
- ความแตกต่างระหว่าง if-else กับ try-catch

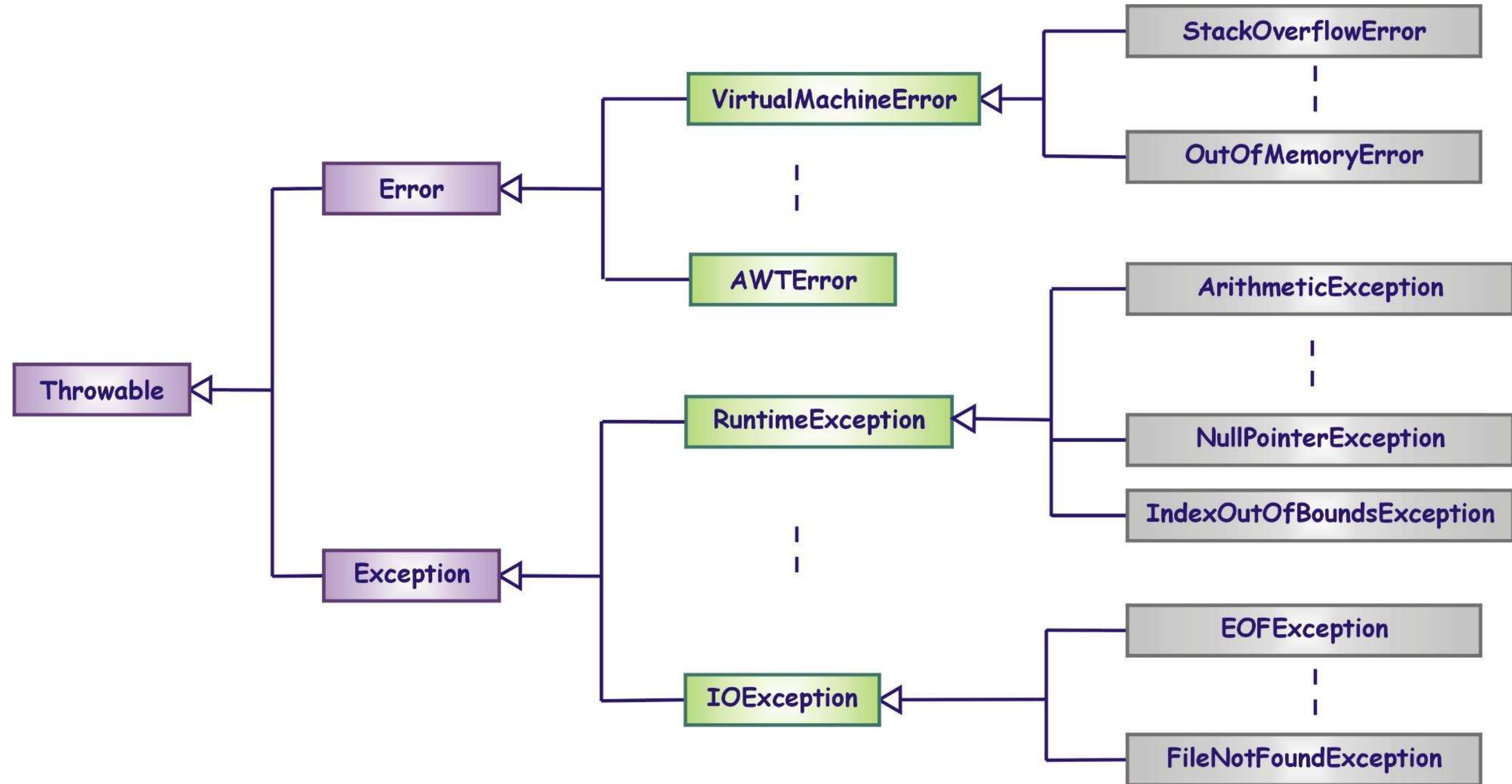
หัวข้อ

- ข้อผิดพลาดคืออะไร
- การสร้างคลาสประเภทข้อผิดพลาดขึ้นมาใหม่
- วิธีการจัดการข้อผิดพลาด
- หลักการจัดการกับข้อผิดพลาด
- ความแตกต่างระหว่าง if-else กับ try-catch

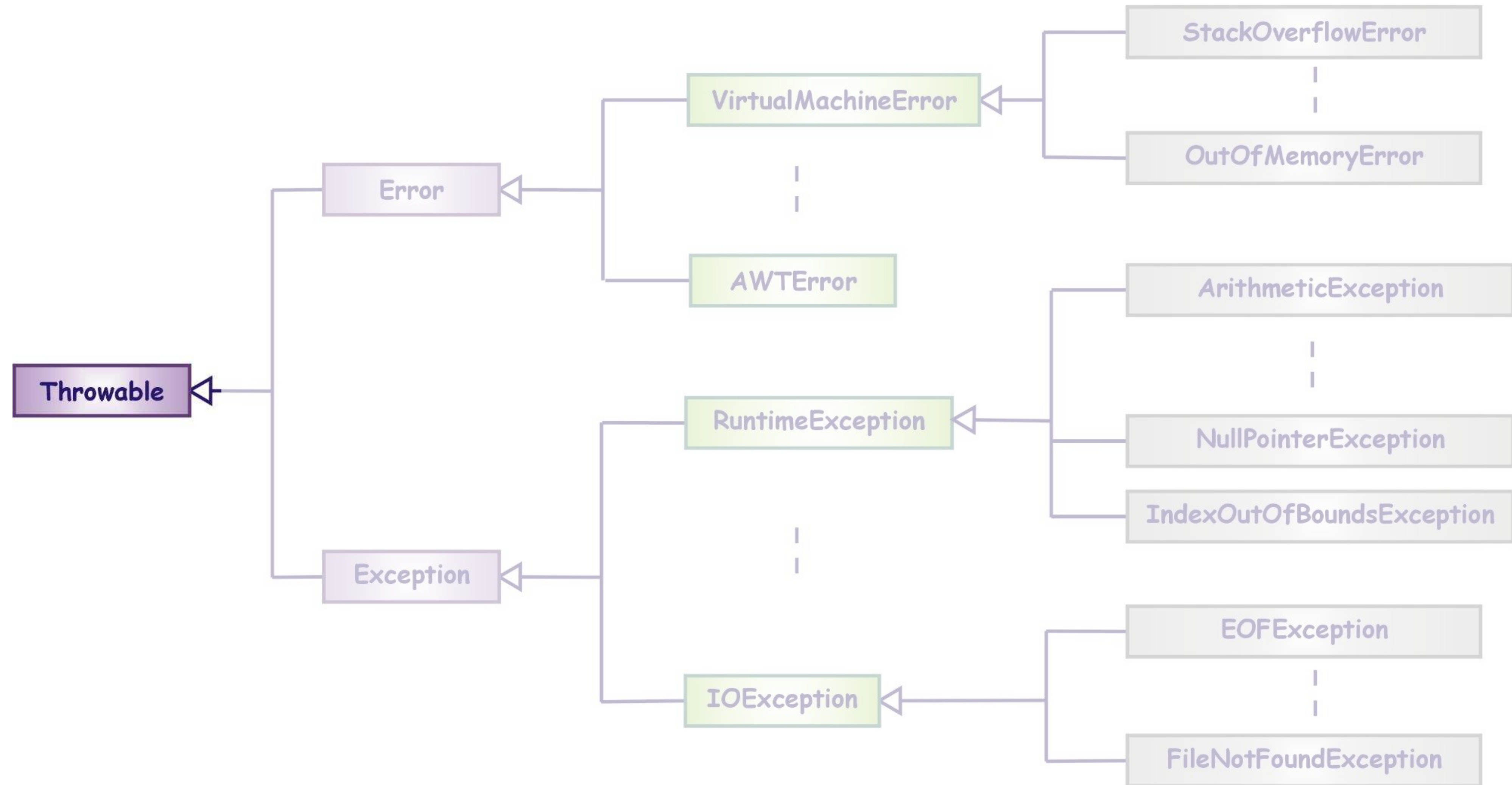
ข้อผิดพลาด

- ข้อผิดพลาดที่อาจเกิดขึ้นขณะรันโปรแกรม แบ่งเป็น 2 ประเภท
 - Error** เป็นข้อผิดพลาดที่~~ไม่~~สามารถแก้ไขและจัดการได้ ส่วนมากเกิดจากการขาดแคลนทรัพยากรของคอมพิวเตอร์ อาทิเช่น system crash หรือ out of memory โดยส่วนมากจะเกิดในสถานะ Runtime เช่น *VirtualMachineError, OutOfMemoryError*
 - Exception** เป็นข้อผิดพลาดที่สามารถแก้ไขหรือจัดการได้ ส่วนมากจะเกิดจากโค้ดของนักพัฒนาโปรแกรม พบมากในสถานะ runtime และ compile time เช่น *FileNotFoundException, ArrayIndexOutOfBoundsException*
- ข้อผิดพลาดจะกำหนดเป็นออบเจ็คของคลาสต่าง ๆ โดยมีคลาส **Throwable** เป็นคลาสราก

คลาสที่เกี่ยวข้องกับข้อผิดพลาด



คลาสที่เกี่ยวข้องกับข้อผิดพลาด



คลาสประเภท Throwable



Constructor Summary

Constructors

Modifier	Constructor	Description
	<code>Throwable()</code>	Constructs a new throwable with <code>null</code> as its detail message.
	<code>Throwable(String message)</code>	Constructs a new throwable with the specified detail message.
	<code>Throwable(String message, Throwable cause)</code>	Constructs a new throwable with the specified detail message and cause.
<code>protected</code>	<code>Throwable(String message, Throwable cause, boolean enableSuppression, boolean writableStackTrace)</code>	Constructs a new throwable with the specified detail message, cause, suppression enabled or disabled, and writable stack trace enabled or disabled.
	<code>Throwable(Throwable cause)</code>	Constructs a new throwable with the specified cause and a detail message of <code>(cause==null ? null : cause.toString())</code> (which typically contains the class and detail message of cause).

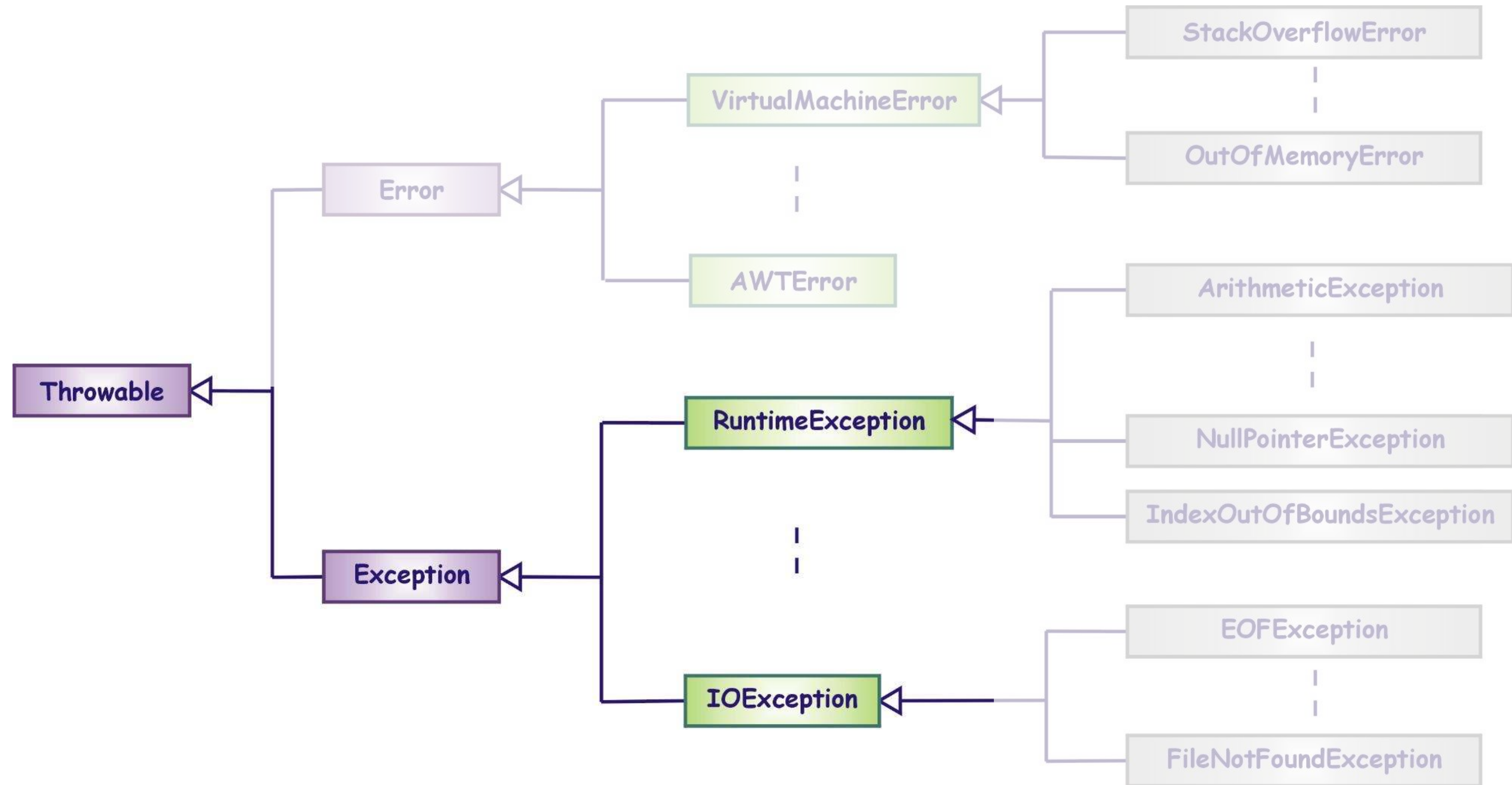
คลาสประเภท Throwable



Method Summary

All Methods	Instance Methods	Concrete Methods
Modifier and Type	Method	Description
final void	<code>addSuppressed(Throwable exception)</code>	Appends the specified exception to the exceptions that were suppressed in order to deliver this exception.
Throwable	<code>fillInStackTrace()</code>	Fills in the execution stack trace.
Throwable	<code>getCause()</code>	Returns the cause of this throwable or null if the cause is nonexistent or unknown.
String	<code>getLocalizedMessage()</code>	Creates a localized description of this throwable.
String	<code>getMessage()</code>	Returns the detail message string of this throwable.
StackTraceElement[]	<code>getStackTrace()</code>	Provides programmatic access to the stack trace information printed by <code>printStackTrace()</code> .
final Throwable[]	<code>getSuppressed()</code>	Returns an array containing all of the exceptions that were suppressed, typically by the try-with-resources statement, in order to deliver this exception.
Throwable	<code>initCause(Throwable cause)</code>	Initializes the <i>cause</i> of this throwable to the specified value.
void	<code>printStackTrace()</code>	Prints this throwable and its backtrace to the standard error stream.
void	<code>printStackTrace(PrintStream s)</code>	Prints this throwable and its backtrace to the specified print stream.
void	<code>printStackTrace(PrintWriter s)</code>	Prints this throwable and its backtrace to the specified print writer.
void	<code>setStackTrace(StackTraceElement[] stackTrace)</code>	Sets the stack trace elements that will be returned by <code>getStackTrace()</code> and printed by <code>printStackTrace()</code> and related methods.
String	<code>toString()</code>	Returns a short description of this throwable.

คลาสที่เกี่ยวข้องกับข้อผิดพลาด



คลาสประเภท Exception

Exception เป็นข้อผิดพลาดที่**เกิดในขณะรันโปรแกรม**ภาษาจาวา ซึ่งสามารถแบ่งออกเป็น 2 ประเภท คือ

- RuntimeException

เป็นข้อผิดพลาดที่อาจ**หลีกเลี่ยงได้**หากมีการเขียนโปรแกรมที่ถูกต้อง

- IOException

เป็นข้อผิดพลาดที่ภาษาจาวากำหนดให้**ต้องมีการจัดการ** หากมีการเรียกใช้เมธอดที่**อาจเกิดข้อผิดพลาดประเภทนี้ได้**

คลาสประเภท Exception



Constructor Summary

Constructors

Modifier	Constructor	Description
	<code>Exception()</code>	Constructs a new exception with <code>null</code> as its detail message.
	<code>Exception(String message)</code>	Constructs a new exception with the specified detail message.
	<code>Exception(String message, Throwable cause)</code>	Constructs a new exception with the specified detail message and cause.
<code>protected</code>	<code>Exception(String message, Throwable cause, boolean enableSuppression, boolean writableStackTrace)</code>	Constructs a new exception with the specified detail message, cause, suppression enabled or disabled, and writable stack trace enabled or disabled.
	<code>Exception(Throwable cause)</code>	Constructs a new exception with the specified cause and a detail message of <code>(cause==null ? null : cause.toString())</code> (which typically contains the class and detail message of cause).

คลาสประเภท Exception ที่สำคัญและพบบ่อย

- ArithmeticException
- ArrayIndexOutOfBoundsException
- EOFException
- FileNotFoundException
- InterruptedException
- IOException
- NullPointerException
- NumberFormatException

ตัวอย่างการเกิด ArithmeticException

```
public class ExceptionArithmeticDemo {  
    public static void main(String args[]) {  
        int a = 1;  
        int b = 0;  
        System.out.println("a/b = " + a/b);  
    }  
}
```

คำนวณ

$\frac{0}{0}$ $\frac{9}{0}$ $\frac{1}{0}$ $\frac{2}{0}$

```
Output - Lab10 (run) x  
run:  
Exception in thread "main" java.lang.ArithmeticException: / by zero  
    at lecture.ExceptionArithmeticDemo.main(ExceptionArithmeticDemo.java:8)  
C:\Users\supan\AppData\Local\NetBeans\Cache\8.2\executor-snippets\run.xml:53: Java returned: 1  
BUILD FAILED (total time: 0 seconds)
```

ตัวอย่างการเกิด NumberFormatException

เปลี่ยน DataType ให้

```
public class ExceptionNumberFormatDemo {  
    public static void main(String args[]) {  
        int a = Integer.parseInt(args[0]);  
        int b = Integer.parseInt(args[1]);  
        System.out.println("a+b = " + (a+b));  
    }  
}
```

"ant" → X

.java → .class

Command Prompt

C:\Projects>javac ExceptionNumberFormatDemo.java

C:\Projects>java ExceptionNumberFormatDemo 1 2

a+b = 3

C:\Projects>java ExceptionNumberFormatDemo 1 John

Exception in thread "main" java.lang.NumberFormatException: For input string: "John"
 at java.lang.NumberFormatException.forInputString(Unknown Source)
 at java.lang.Integer.parseInt(Unknown Source)
 at java.lang.Integer.parseInt(Unknown Source)
 at ExceptionNumberFormatDemo.main(ExceptionNumberFormatDemo.java:5)

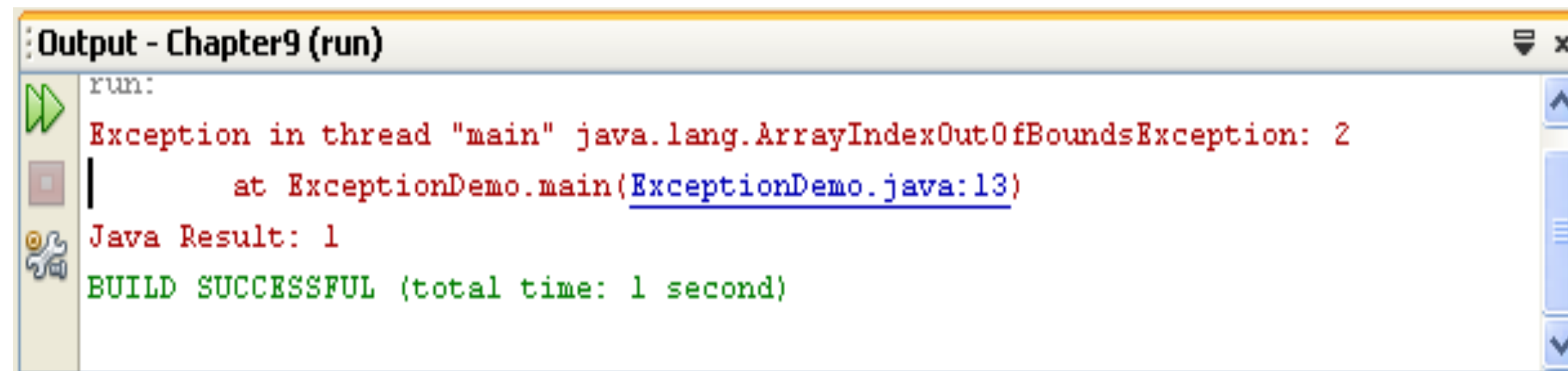
C:\Projects>

ตัวอย่างการเกิด ArrayIndexOutOfBoundsException



เกิดจากอะไร

```
public class ExceptionDemo {  
    public static void main(String args[]) {  
        int [] num = {10,20};  
        System.out.println(num[2]);  
        System.out.println("Hello");  
    }  
}
```



The screenshot shows an IDE output window titled "Output - Chapter9 (run)". It displays the following text:

```
run:  
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 2  
    at ExceptionDemo.main(ExceptionDemo.java:13)  
Java Result: 1  
BUILD SUCCESSFUL (total time: 1 second)
```

The exception message indicates that an `ArrayIndexOutOfBoundsException` occurred at index 2 of the array in `ExceptionDemo.java` at line 13. The build was successful, but the program terminated due to the exception.

ตัวอย่างการเกิด NullPointerException

```
public class ExceptionNullPointer {  
    private Student s1;  
    public void testNull() {  
        System.out.println("Name: " + s1.getName());  
    }  
    public static void main(String[] args) {  
        new ExceptionNullPointer().testNull();  
    }  
}
```

- ① ประกาศ obj
- ② ขาดการ init → s1 = new Student();
- ③ เรียกใช้งาน

Output - Lab10 (run) ×

```
run:  
Exception in thread "main" java.lang.NullPointerException  
    at lecture.ExceptionNullPointer.testNull(ExceptionNullPointer.java:21)  
    at lecture.ExceptionNullPointer.main(ExceptionNullPointer.java:25)  
C:\Users\supan\AppData\Local\NetBeans\Cache\8.2\executor-snippets\run.xml:53: Java returned: 1  
BUILD FAILED (total time: 0 seconds)
```

หัวข้อ

- ข้อผิดพลาดคืออะไร
- การสร้างคลาสประเภทข้อผิดพลาดขึ้นมาใหม่
- วิธีการจัดการข้อผิดพลาด
- หลักการจัดการกับข้อผิดพลาด
- ความแตกต่างระหว่าง if-else กับ try-catch

การสร้างคลาสประเภท Exception ขึ้นใหม่

การสร้างคลาสประเภท Exception ขึ้นใหม่ในภาษาจาวา จะเรียกว่า “custom exception” หรือ “user-defined exception” นิยมนำมาใช้สำหรับปรับแต่ง Exception ตามความต้องการของผู้ใช้ อย่างไรก็ตาม บางครั้งเราจำเป็นต้องสร้าง Custom Exception ขึ้น เนื่องจาก

- เพื่อตรวจจับและจัดเตรียมวิธีการรับมือเฉพาะด้านกับ Exception ที่มีความจำเพาะ
- Business logic exceptions คือ Exception ที่เกี่ยวข้องกับ Business logic หรือกระบวนการทำงานทางธุรกิจเพื่อเป็นประโยชน์สำหรับนักพัฒนาในการทำความเข้าใจปัญหาที่แท้จริง

การสร้างคลาสประเภท Exception ขึ้นใหม่

การสร้างคลาสประเภท Exception ขึ้นมาใหม่ สามารถทำได้โดยนิยามคลาสใด ๆ ให้สืบทอดมาจากคลาสที่ชื่อ Exception โดยทั่วไป คลาสที่ชื่อ Exception จะมี constructor สองรูปแบบคือ

- `public Exception()`
- `public Exception(String s)*`

ดังนั้น คลาสที่สืบทอดมาจากคลาสที่ชื่อ Exception **ควรมี constructor ทั้งสองแบบ** โดยรูปแบบหนึ่งจะมี argument ที่มีชนิดข้อมูลเป็น String และมีคำสั่งแรกใน constructor เป็นคำสั่ง `super(s);`

```
public class MyOwnException extends Exception {  
    public MyOwnException (String s) {  
        super(s);  
    }  
}
```

Handwritten annotations: An orange box highlights `extends Exception`. An arrow points from the `s` in `String s` to the `s` in `super(s)`, with a circled '3' next to it. Another arrow points from the `s` in `String s` to the `s` in `super(s)`, with a circled '2' next to it.

การเขียนเมธอดเพื่อส่งออกเจ็คประเภท Exception



เมธอดที่ต้องการส่งออกเจ็คประเภท Exception เมื่อเกิดข้อผิดพลาดขึ้นในคำสั่งใด จะต้องเรียกใช้คำสั่งที่ชื่อ **throw** เพื่อจะสร้างออกเจ็คของคลาสประเภท Exception ขึ้นมา

รูปแบบ

```
throw new ExceptionType ( [arguments] )
```

นอกจากนี้ คำสั่งประกาศเมธอดนั้นจะต้องมีคำสั่ง **throws** เพื่อกำหนดให้คำสั่งในเมธอดอื่น ๆ ที่เรียกใช้เมธอดนี้ต้องเขียนคำสั่งในการจัดการกับข้อผิดพลาดนี้

ตัวอย่างคลาส FileHandler

```
import java.io.*;
```

```
public class FileHandler {  
    public static void openFile(String filename) throws MyOwnException {  
        File f = new File(filename);  
        if (!f.exists()) {  
            throw new MyOwnException("File Not Found");  
        }  
    }  
}
```

```
public class MyOwnException extends Exception {  
    public MyOwnException(String s) {  
        super(s);  
    }  
}
```

ป้อนค่า object

ขอมอบ exception หน่อย

throw → throws

ตัวอย่างโปรแกรมที่มีการจัดการกับข้อผิดพลาด



```
public class FileOpener {  
    public static void main (String args[]) {  
        try {  
            FileHandler.openFile(args[0]);  
            System.out.println("Open successful");  
        } catch (MyOwnException ex) {  
            System.err.println(ex);  
        }  
    }  
}
```

หัวข้อ



- ข้อผิดพลาดคืออะไร
- การสร้างคลาสประเภทข้อผิดพลาดขึ้นมาใหม่
- วิธีการจัดการข้อผิดพลาด
- หลักการการจัดการกับข้อผิดพลาด
- ความแตกต่างระหว่าง if-else กับ try-catch

การจัดการกับ Exception

การจัดการกับ Exception มี 2 แบบ คือ

- ใช้คำสั่ง **try-catch**

แบบนี้ง่าย ๆ

- ใช้คำสั่ง **throws** ในการประกาศเมธอดที่จะมีการเรียกใช้เมธอดใด ๆ ที่อาจส่งออกไปเจ็คประเภท Exception ซึ่งแบ่งได้เป็น 3 แบบ
 - Throwing
 - Rethrowing
 - Wrapping

การจัดการกับ Exception

การจัดการกับ Exception มี 2 แบบ คือ

- ใช้คำสั่ง **try-catch**
- ใช้คำสั่ง **throws** ในการประกาศเมธอดที่จะมีการเรียกใช้เมธอดใด ๆ ที่อาจส่งออกไปเจ็คประเภท Exception ซึ่งแบ่งได้เป็น 3 แบบ
 - Throwing
 - Rethrowing
 - Wrapping

คำสั่ง try...catch

ภาษาจาวามีคีย์เวิร์ด try ที่เป็นคำสั่งที่ใช้ในการจัดการกับเมธอดหรือคำสั่งที่อาจเกิดข้อผิดพลาดซึ่งจะส่งออกไปแจ้งประเภท Exception ในขณะรันโปรแกรม

รูปแบบ

```
try {  
    [statements]  
}
```

โปรแกรมภาษาจาวาจะสั่งงานชุดคำสั่งที่อยู่ในบล็อกที่ละคำสั่ง และหากเกิดข้อผิดพลาดขึ้นในคำสั่งประเภทใดก็จะมีคำสั่ง ออกไปแจ้งของข้อผิดพลาดประเภท Exception นั้นขึ้นมา

คำสั่ง `try...catch`

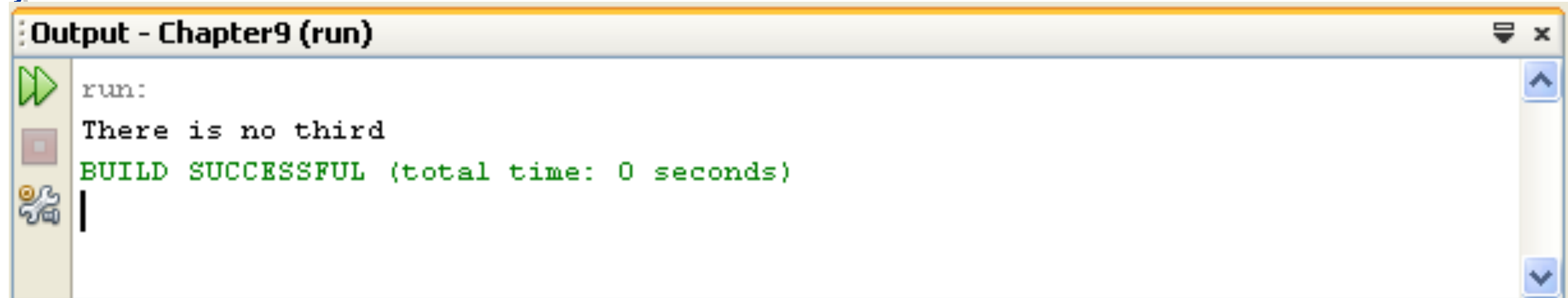
ในกรณีที่ต้องการจัดการกับข้อผิดพลาดที่เกิดขึ้น โปรแกรมจะต้องมีชุดคำสั่งอยู่ในบล็อกของคีย์เวิร์ด `catch` ที่จะระบุชนิดของข้อผิดพลาดในคลาสประเภท `Exception` ที่ต้องการจัดการ

รูปแบบ

```
catch (ExceptionType argumentName) {  
    [statements]  
}
```


ตัวอย่างโปรแกรมที่มีการจัดการกับข้อผิดพลาด

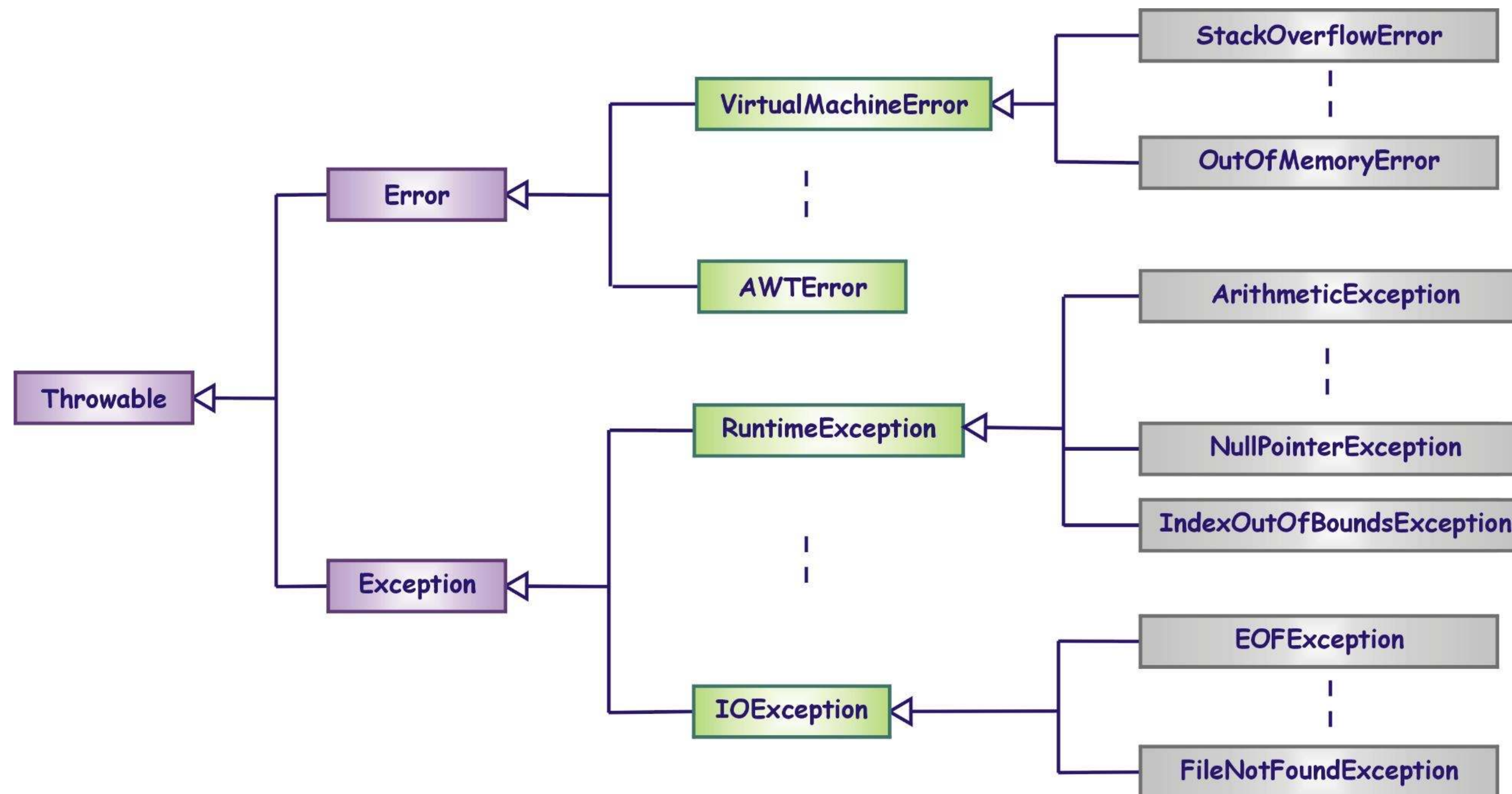
```
public class ExceptionHandlingDemo {  
    public static void main(String args[]) {  
        int [] num = {10,20};  
        try {  
            System.out.println(args[2]);  
        } catch (ArrayIndexOutOfBoundsException ex) {  
            System.out.println("There is no third");  
        }  
    }  
}
```



```
Output - Chapter9 (run)  
run:  
There is no third  
BUILD SUCCESSFUL (total time: 0 seconds)
```


การจัดการกับข้อผิดพลาด

เราสามารถจัดการกับข้อผิดพลาดของ Exception โดยใช้คลาสที่เป็น superclass ของ Exception นั้นได้
อาทิเช่น เราสามารถจัดการกับ FileNotFoundException โดยใช้ IOException หรือ Exception แทนได้



การจัดการกับข้อผิดพลาดหลาย ๆ ประเภท

- โปรแกรมภาษาจาวาสามารถจะมีชุดคำสั่งของ **บล็อก catch** ได้มากกว่าหนึ่งชุดสำหรับใน **แต่** **ละบล็อกคำสั่ง try**
- ชนิดของอุปเจ็คประเภท Exception ที่อยู่ในชุดคำสั่งของบล็อก catch **จะต้องเรียง** **ตามลำดับการสืบทอด**
- ในกรณีที่มีข้อผิดพลาดเกิดขึ้น ภาษาจาวาจะพิจารณาว่าเป็นข้อผิดพลาดชนิดใด ซึ่งการที่จะจัดการกับอุปเจ็คประเภท Exception นั้นจะพิจารณาจากคลาสที่มีการสืบทอดตามลำดับชั้น

การจัดการกับข้อผิดพลาดหลาย ๆ ประเภท

โปรแกรมภาษาจาวาสามารถจะมีชุดคำสั่งของ **บล็อก catch** ได้มากกว่าหนึ่งชุดสำหรับในแต่ละบล็อกคำสั่ง **try**

```
public class SeqExceptionExample {  
    public void testNumber(int a, int b) {  
        try {  
            System.out.println(a/b);  
        } catch (ArithmeticException e) {  
            e.printStackTrace();  
        } catch (Exception e) {  
            e.printStackTrace();  
        }  
    }  
    public static void main(String[] args) {  
        new SeqExceptionExample().testNumber(5, 0);  
    }  
}
```

```
java.lang.ArithmeticException: / by zero  
    at TryCatchProject.SeqExceptionExample.testNumber(SeqExceptionExample.java:15)  
    at TryCatchProject.SeqExceptionExample.main(SeqExceptionExample.java:23)
```


การจัดการกับข้อผิดพลาดหลาย ๆ ประเภท

ชนิดของอุปเจ็คประเภท Exception ที่อยู่ในชุดคำสั่งของบล็อก catch **จะต้องเรียงตามลำดับการสืบทอด**

```
public class SeqExceptionExample {  
    public void testNumber(int a, int b) {  
        try{  
            System.out.println(a/b);  
        } catch (Exception e) {  
            e.printStackTrace();  
        } catch (ArithmeticException e) {  
            e.printStackTrace();  
        }  
    }  
    public static void main(String[] args) {  
        new SeqExceptionExample().testNumber(5, 0);  
    }  
}
```

```
Exception in thread "main" java.lang.RuntimeException: Uncompilable source code -  
exception java.lang.ArithmeticException has already been caught  
    at TryCatchProject.SeqExceptionExample.testNumber(SeqExceptionExample.java:14)  
    at TryCatchProject.SeqExceptionExample.main(SeqExceptionExample.java:24)
```


การจัดการกับข้อผิดพลาดหลาย ๆ ประเภท



ในกรณีที่มีข้อผิดพลาดเกิดขึ้น ภาษาจาวาจะพิจารณาว่าเป็นข้อผิดพลาดชนิดใด ซึ่งการที่จะจัดการกับข้อผิดพลาดประเภท Exception นั้นจะพิจารณาจากคลาสที่มีการสืบทอดตามลำดับชั้น

```
public class ExceptionHandlingDemoV2 {  
    public static void main(String args[]) {  
        try {  
            int i = Integer.parseInt(new Scanner(System.in).nextLine());  
            System.out.println(4 / i);  
        } catch (ArithmeticException ex) {  
            System.out.println(ex.toString());  
        } catch (NumberFormatException ex) {  
            System.out.println("Invalid numeric format");  
        }  
    }  
}
```

ตัวอย่างโปรแกรมที่ไม่ถูกต้อง

```
public class ExceptionHandlingDemoV3 {  
    public static void main(String args[]) {  
        try {  
            int i = Integer.parseInt(args[0]);  
            System.out.println(4 / i);  
            System.out.println(args[2]);  
        } catch (RuntimeException ex) {  
            System.out.println(ex.toString());  
        } catch (ArrayIndexOutOfBoundsException ex) {  
            System.out.println("There is no third command line argument");  
        }  
    }  
}
```

Class ArrayIndexOutOfBoundsException

```
java.lang.Object  
    java.lang.Throwable  
        java.lang.Exception  
            java.lang.RuntimeException  
                java.lang.IndexOutOfBoundsException  
                    java.lang.ArrayIndexOutOfBoundsException
```

บล็อก finally

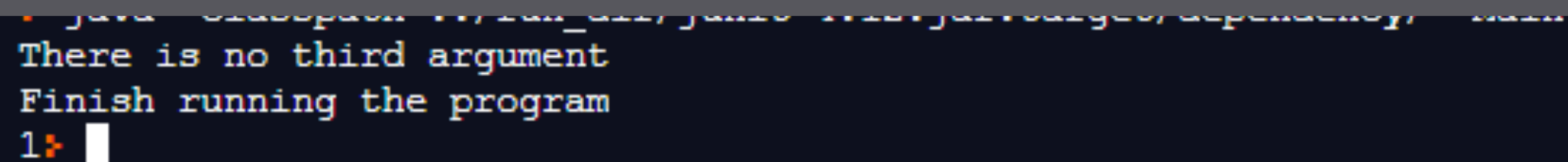
- ภาษาจาวามีคีย์เวิร์ด **finally** ที่จะมีชุดคำสั่งอยู่ในบล็อกเพื่อระบุให้โปรแกรมทำชุดคำสั่งดังกล่าวหลังจากสิ้นสุดการทำงานของชุดคำสั่งในบล็อก **try** หรือ **catch**
- ภาษาจาวา**จะทำชุดคำสั่งในบล็อก finally เสมอ** แม้ว่าจะมีคำสั่ง **return** ในบล็อก **try** หรือ **catch** ก่อนก็ตาม
- กรณีเดียวที่**จะไม่ทำชุดคำสั่งในบล็อก finally** คือมีคำสั่ง **System.exit();**

ตัวอย่างโปรแกรม

```
public class FinallyDemo {  
    public static void main(String args[]) {  
        try {  
            System.out.println(args[2]);  
            System.out.println("Hello");  
        } catch (ArrayIndexOutOfBoundsException ex) {  
            System.out.println("There is no third argument");  
        } finally {  
            System.out.println("Finish running the program");  
        }  
    }  
}
```


ตัวอย่างโปรแกรม

```
public class Main {  
    public static void main(String args[]) {  
        Main obj = new Main();  
        System.out.println(obj.myMethod(args));  
    }  
    public int myMethod(String args[]) {  
        try {  
            System.out.println(args[2]);  
            return 0;  
        } catch (ArrayIndexOutOfBoundsException ex) {  
            System.out.println("There is no third argument");  
        } finally {  
            System.out.println("Finish running the program");  
            return 1;  
        }  
    }  
}
```



```
java -classpath .; run-err; junit.jar org.apache.commons.lang3.StringUtils main  
There is no third argument  
Finish running the program  
1>
```

ตัวอย่างโปรแกรม

```
public class Main {  
    public static void main(String[] args) {  
        System.out.print(test(0));  
    }  
    public static int test(int b){  
        try {  
            System.out.println("in test : " + 1/b);  
            return 0;  
        } catch (Exception e) {  
            System.out.println("catch");  
            return 1;  
        } finally {  
            System.out.println("final");  
        }  
    }  
}
```

ผลลัพธ์

catch

final

1

ตัวอย่างโปรแกรม

```
public class Main {  
    public static void main(String[] args) {  
        System.out.print(test(1));  
    }  
    public static int test(int b){  
        try {  
            System.out.println("in test : " + 1/b);  
            return 0;  
        } catch (Exception e) {  
            System.out.println("catch");  
            return 1;  
        } finally {  
            System.out.println("final");  
        }  
    }  
}
```

ผลลัพธ์

in test : 1

final

0

ตัวอย่างโปรแกรม

```
public class Main {  
    public static void main(String[] args) {  
        System.out.print(test(0));  
    }  
    public static int test(int b){  
        try {  
            System.out.println("in test : " + 1/b);  
            return 0;  
        } catch (Exception e) {  
            System.out.println("catch");  
            return 1;  
        } finally {  
            System.out.println("final");  
            return 2;  
        }  
    }  
}
```

ผลลัพธ์

catch

final

2

ตัวอย่างโปรแกรม

```
public class Main {  
    public static void main(String[] args) {  
        System.out.print(test(1));  
    }  
    public static int test(int b){  
        try {  
            System.out.println("in test : " + 1/b);  
            return 0;  
        } catch (Exception e) {  
            System.out.println("catch");  
            return 1;  
        } finally {  
            System.out.println("final");  
            return 2;  
        }  
    }  
}
```

ผลลัพธ์

in test : 1

final

2

การ Union ของคำสั่ง Try-Catch

เพื่อที่จะลดโค้ดที่ซ้ำซ้อนหรือเหมือนกันลง ตั้งแต่ **jdk 1.7** เป็นต้นมา จะสามารถรวมโค้ดในส่วนของ catch ที่มีการทำงานเหมือนกันเข้ามาไว้ใน catch อันเดียวกันได้ โดยอาศัยตัวดำเนินการ | ดังแสดงในตัวอย่าง

```
public class TestUnionCode {  
    public void testCode(String filePath) {  
        try {  
            // some code  
        } catch (IOException | NumberFormatException ex) {  
            // handle  
        }  
    }  
}
```

การใช้ try-with-resources

ตั้งแต่ jdk 1.7 เป็นต้นมา การจัดการทรัพยากรที่จำเป็นต้องปิดหรือคืนทรัพยากรนั้น สามารถใช้งาน **try-with-resources** ได้ ซึ่ง try-with-resources จะช่วยอำนวยความสะดวกเกี่ยวกับการเชื่อมต่อและ/หรือการคืนทรัพยากรให้กับระบบ โดยที่เมื่อสิ้นสุดการทำงานภายใต้ try-with-resources แล้ว ทรัพยากรเหล่านั้นจะถูกคืนให้ระบบโดยอัตโนมัติ โดยไม่จำเป็นต้องเขียน close อีกต่อไป

```
try (/* resources declaration */ ) {  
  
    /* statement block */  
  
}
```


ตัวอย่างโปรแกรมโดยอาศัย Try-Catch

```
public class TryResourceDemo {
    public static void main(String args[]) {
        Connection con = null;
        Statement stm = null;
        ResultSet rs = null;
        try {
            String sql = "SELECT * FROM User";
            con = getConnection();
            stm = con.createStatement();
            rs = stm.executeQuery(sql);
            while(rs.next()) {
                rs.getString("USERNAME");
                rs.getString("EMAIL");
            }
        } catch (SQLException sqle) {

        } finally {
            closeResource(rs, stm, con);
        }
    }
}
```

```
public static void closeResource (ResultSet
rs, Statement stm, Connection con) {
    if (rs != null) {
        try {
            rs.close();
        } catch (SQLException e) {}
    }
    if (stm != null) {
        try {
            stm.close();
        } catch (SQLException e) {}
    }
    if (con != null) {
        try {
            con.close();
        } catch (SQLException e) {}
    }
}
}
```

ตัวอย่างโปรแกรมโดยอาศัย try-with-resources



```
String sql = "SELECT * FROM UserAccount";  
try (Connection con = getConnection();  
    Statement stm = con.createStatement();  
    ResultSet rs = stm.executeQuery(sql);) {  
  
    while(rs.next()) {  
        rs.getString("USERNAME");  
        rs.getString("EMAIL");  
    }  
  
} catch (SQLException e) {  
  
}
```

อ้างอิง : <https://www.lordgift.in.th/2017/01/java-try-with-resouces-java-7.html>

อย่างไรก็ตาม วัตถุที่สร้างในวงเล็บของ Try ต้องเป็นคลาสที่สืบทอดมาจากอินเตอร์เฟส AutoCloseable เท่านั้น และในวงเล็บของ Try ต้องเป็นการประกาศและสร้างวัตถุเท่านั้น

อ้างอิง : <https://docs.oracle.com/javase/8/docs/api/java/lang/AutoCloseable.html>

การจัดการกับ Exception

การจัดการกับ Exception มี 2 แบบ คือ

- ใช้คำสั่ง **try-catch**
- ใช้คำสั่ง **throws** ในการประกาศเมธอดที่จะมีการเรียกใช้เมธอดใด ๆ ที่อาจส่งออกไปเจ็คประเภท Exception ซึ่งแบ่งได้เป็น 3 แบบ
 - Throwing
 - Rethrowing
 - Wrapping

คำสั่ง throws

รูปแบบการใช้คำสั่ง throws มีดังนี้

```
[modifier] return_type methodName ([arguments]) throws  
    ExceptionType [,ExceptionType2] {  
    ...  
}
```

ตัวอย่าง

```
public void openfile(String s) throws FileNotFoundException {  
    ...  
}
```

คำสั่ง throws

เมธอดใด ๆ สามารถที่จะจัดการกับ Exception โดยใช้คำสั่ง **throws** ได้มากกว่าหนึ่งประเภท

ตัวอย่าง

```
public void openFile(String s) throws  
    FileNotFoundException, UnknownHostException {  
    ...  
}
```

กรณีที่มีการใช้คำสั่ง throws ส่งต่อไปเรื่อย ๆ แล้วเมธอด main() ซึ่งเรียกใช้เมธอดสุดท้ายที่ใช้คำสั่ง throws ไม่มีการจัดการกับออกเจ็คประเภท Exception ดังกล่าว **โปรแกรมจะเกิดข้อผิดพลาดในขั้นตอนการรันโปรแกรม** เมื่อมีข้อผิดพลาดของออกเจ็คประเภท Exception ดังกล่าวเกิดขึ้น

ตัวอย่างโปรแกรมที่ไม่มีการจัดการกับ Exception

```
public class ExceptionDemo1 {  
    public static void main(String args[]) {  
        ExceptionDemo1 ex1 = new ExceptionDemo1();  
        ex1.method1();  
    }  
    public void method1() throws ArithmeticException {  
        method2();  
    }  
    public void method2() throws ArithmeticException {  
        System.out.println(2/0);  
    }  
}
```

Output - Chapter9 (run)

run:
Exception in thread "main" java.lang.ArithmeticException: / by zero
 at ExceptionDemo1.method2(ExceptionDemo1.java:22)
 at ExceptionDemo1.method1(ExceptionDemo1.java:18)
 at ExceptionDemo1.main(ExceptionDemo1.java:14)
Java Result: 1
BUILD SUCCESSFUL (total time: 2 seconds)

หลักการ Rethrowing และ Wrapping

การ throwing คือการส่งต่อออกเจ็คประเภท Exception ออกไปสู่ผู้ที่เรียกใช้ให้เป็นคนจัดการกับข้อผิดพลาด ด้วยคำสั่ง throws แล้วยังสามารถทำได้อีก 2 แบบ ได้แก่

- Rethrowing คือ การใช้งาน try-catch ให้ดักจับออกเจ็คประเภท Exception ที่เกิดขึ้นและส่งต่อออกไปด้วย throws อีกทีหนึ่ง **โดยไม่มีการเปลี่ยนแปลงประเภทของออกเจ็ค Exception นั้น**
- Wrapping คือ การใช้งาน try-catch ให้ดักจับออกเจ็คประเภท Exception ที่เกิดขึ้นและส่งต่อออกไปด้วย throws อีกทีหนึ่ง **โดยมีการเปลี่ยนแปลงประเภทของออกเจ็ค Exception นั้นให้มีความจำเพาะหรือชัดเจนยิ่งขึ้น**

```
public class ForgotToDeclareComponentException extends Exception {  
    public ForgotToDeclareComponentException(String name) {  
        super(name) ;  
    }  
}
```


หลักการ Rethrowing และ Wrapping

```
public class WrappingExample {  
    public JFrame fr; public JButton btn;  
    public WrappingExample() throws ForgotToDeclareComponentException, Exception {  
        try{  
            fr = new JFrame();  
            fr.add(btn);  
        } catch (NullPointerException e) { // Wrapping  
            throw new ForgotToDeclareComponentException("You forgot to declare the component");  
        } catch (Exception e) { throw e; } // Rethrowing  
    }  
    public static void main(String[] args) {  
        try {  
            new WrappingExample();  
        } catch (ForgotToDeclareComponentException ex) {  
            System.out.println(ex);  
            ex.printStackTrace();  
        } catch (Exception ex) {  
            System.out.println(ex);  
            ex.printStackTrace();  
        }  
    }  
}
```

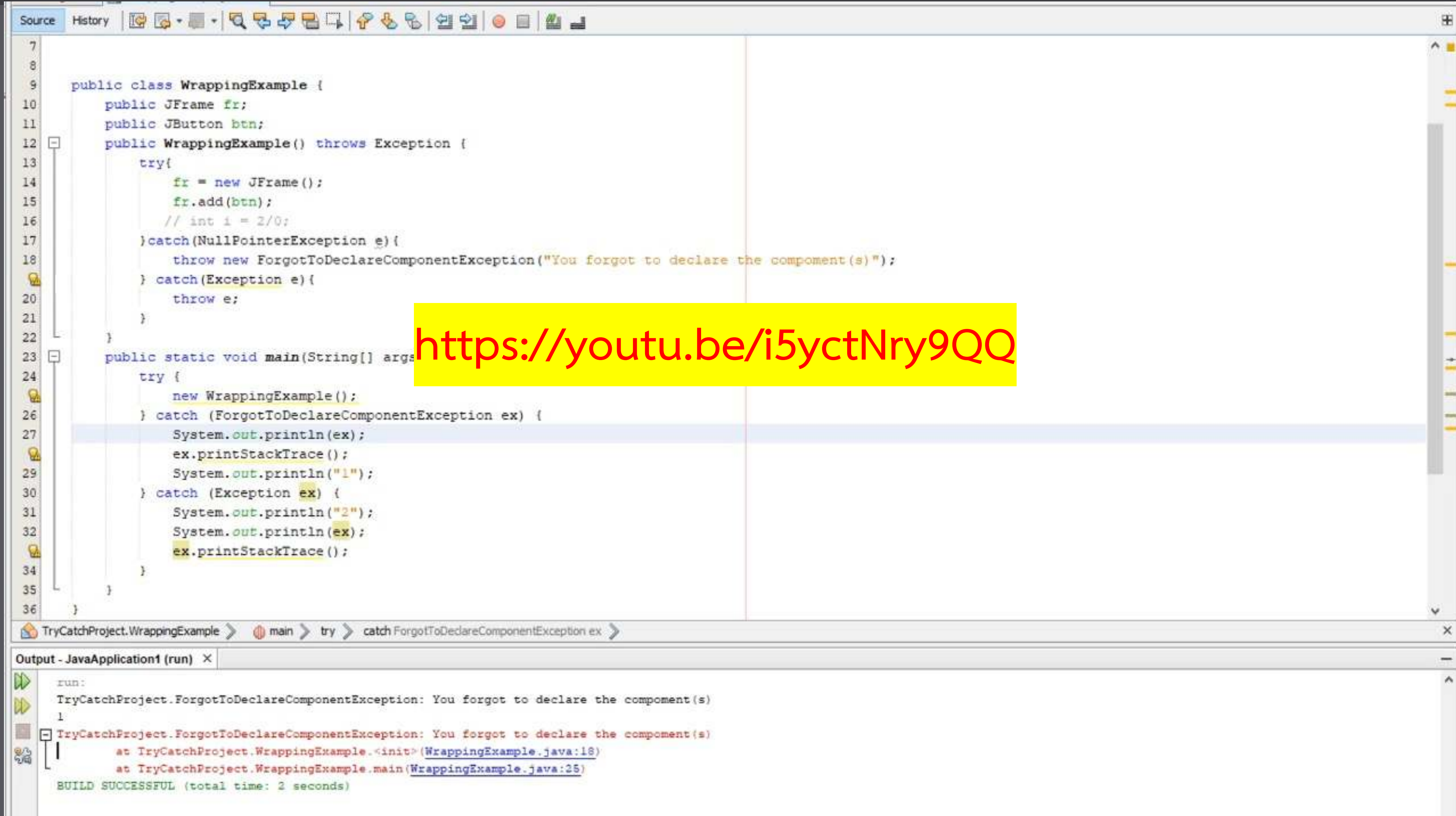
Handwritten notes:

- throws* is underlined in blue.
- ForgotToDeclareComponentException, Exception* is underlined in blue.
- try{* is underlined in blue.
- fr.add(btn);* is underlined in blue.
- NullPointerException e* is underlined in blue.
- Wrapping* is underlined in blue.
- throw new* is underlined in blue.
- ForgotToDeclareComponentException("You forgot to declare the component");* is underlined in blue.
- Exception e* is underlined in blue.
- throw e;* is underlined in blue.
- Rethrowing* is underlined in blue.
- new WrappingExample();* is underlined in blue.
- ForgotToDeclareComponentException ex* is underlined in blue.

Handwritten text:

- null point* with an arrow pointing to *NullPointerException e*.

ข้อสังเกต



The screenshot displays an IDE window with a Java source file named `WrappingExample.java`. The code defines a class `WrappingExample` with a constructor that throws an exception and a `main` method that catches it. The output window shows the execution results, including the exception message and the stack trace.

```
7  
8  
9 public class WrappingExample {  
10     public JFrame fr;  
11     public JButton btn;  
12     public WrappingExample() throws Exception {  
13         try{  
14             fr = new JFrame();  
15             fr.add(btn);  
16             // int i = 2/0;  
17         } catch (NullPointerException e) {  
18             throw new ForgotToDeclareComponentException("You forgot to declare the component(s)");  
19         } catch (Exception e) {  
20             throw e;  
21         }  
22     }  
23     public static void main(String[] args) {  
24         try {  
25             new WrappingExample();  
26         } catch (ForgotToDeclareComponentException ex) {  
27             System.out.println(ex);  
28             ex.printStackTrace();  
29             System.out.println("1");  
30         } catch (Exception ex) {  
31             System.out.println("2");  
32             System.out.println(ex);  
33             ex.printStackTrace();  
34         }  
35     }  
36 }
```

Output - JavaApplication1 (run) X

```
run:  
TryCatchProject.ForgotToDeclareComponentException: You forgot to declare the component(s)  
1  
TryCatchProject.ForgotToDeclareComponentException: You forgot to declare the component(s)  
at TryCatchProject.WrappingExample.<init>(WrappingExample.java:18)  
at TryCatchProject.WrappingExample.main(WrappingExample.java:25)  
BUILD SUCCESSFUL (total time: 2 seconds)
```

<https://youtu.be/i5yctNry9QQ>

ตัวอย่างที่ 1

```
public class InvalidAgeException extends Exception {
    public InvalidAgeException () { super("The age range entered is invalid."); }
    public InvalidAgeException (String str) { super(str); }
}

public class Main {
    public static void validate (int age) throws InvalidAgeException{
        if(age < 18){
            throw new InvalidAgeException("age is not valid to vote");
        } else if (age < 0){
            throw new InvalidAgeException();
        } else {
            System.out.println("welcome to vote");
        }
    }

    public static void main(String args[]) {
        try {
            validate(13);
        }
        catch (InvalidAgeException ex) {
            System.out.println("Exception occurred: " + ex);
        }
    }
}
```



Output - JavaApplication15 (run) ×

run:
Exception occurred: InvalidAgeException: age is not valid to vote
BUILD SUCCESSFUL (total time: 0 seconds)

ตัวอย่างที่ 1

```
public class InvalidAgeException extends Exception {
    public InvalidAgeException () { super("The age range entered is invalid."); }
    public InvalidAgeException (String str) { super(str); }
}

public class Main {
    public static void validate (int age) throws InvalidAgeException{
        if(age < 18){
            throw new InvalidAgeException("age is not valid to vote");
        } else if (age < 0){
            throw new InvalidAgeException();
        } else {
            System.out.println("welcome to vote");
        }
    }

    public static void main(String args[]) {
        try {
            validate(13);
        }
        catch (InvalidAgeException ex) {
            System.out.println("Exception occurred: " + ex);
        }
    }
}
```



Output - JavaApplication15 (run) ×

run:
Exception occurred: InvalidAgeException: age is not valid to vote
BUILD SUCCESSFUL (total time: 0 seconds)

กฎของการกำหนดเมธอดแบบ overridden

อย่างไรก็ตาม เมธอดแบบ overridden จะไม่อนุญาตให้มีการจัดการออกเจ็คประเภท Exception โดยใช้คำสั่ง **throws** มากกว่าที่เมธอดเดิมจัดการอยู่ได้

```
import java.io.*;
public class Main {
    public static void main(String[] args) { new OverrideExceptionV2().myMethods(); }
}
public class Parent {
    public void myMethods() throws FileNotFoundException { }
}
public class OverrideExceptionV2 extends Parent {
    public void myMethods() throws FileNotFoundException, IOException {
        new FileInputStream("temp.txt");
    }
}
```

```
Main.java:16: error: myMethods() in OverrideExceptionV2 cannot override myMethods() in Parent
    public void myMethods() throws FileNotFoundException, IOException {
                ^
    overridden method does not throw IOException
```


หัวข้อ

- ข้อผิดพลาดคืออะไร
- การสร้างคลาสประเภทข้อผิดพลาดขึ้นมาใหม่
- วิธีการจัดการข้อผิดพลาด
- หลักการจัดการกับข้อผิดพลาด
- ความแตกต่างระหว่าง if-else กับ try-catch

ทำไมต้อง Try-Catch แทนที่จะใช้ If-Condition

```
public class Error01{  
    public int [] num = new int[5];  
    public int GetItem(int index){  
        try{  
            return num[index];  
        } catch (ArrayIndexOutOfBoundsException e){  
            return -1;  
        }  
    }  
}
```



ไม่แนะนำ

```
public class Error02{  
    public int [] num = new int[5];  
    public int GetItem(int index){  
        if (index >= 0 && index < num.length)  
            return num[index];  
        else  
            return -1;  
    }  
}
```



แนะนำ

ไม่ควรเลือกใช้ Try-Catch สำหรับจัดการ Exception ในกรณี normal flow of control ถ้าเป็นไปได้ เว้นแต่เกิดจากการทำงานผิดพลาดของระบบ (system failures) หรือการจัดการที่ผิดพลาดในบางกรณีที่ไม่แน่นอน (operations with potential race conditions) อาทิเช่น เราควรเขียนโค้ดเพื่อตรวจสอบลำดับการอ้างอิงถึงสมาชิกใน Array ก่อนแทนที่จะ Try-Catch หรือ Throws ออกเจ็คของ Exception นั้นออกมา

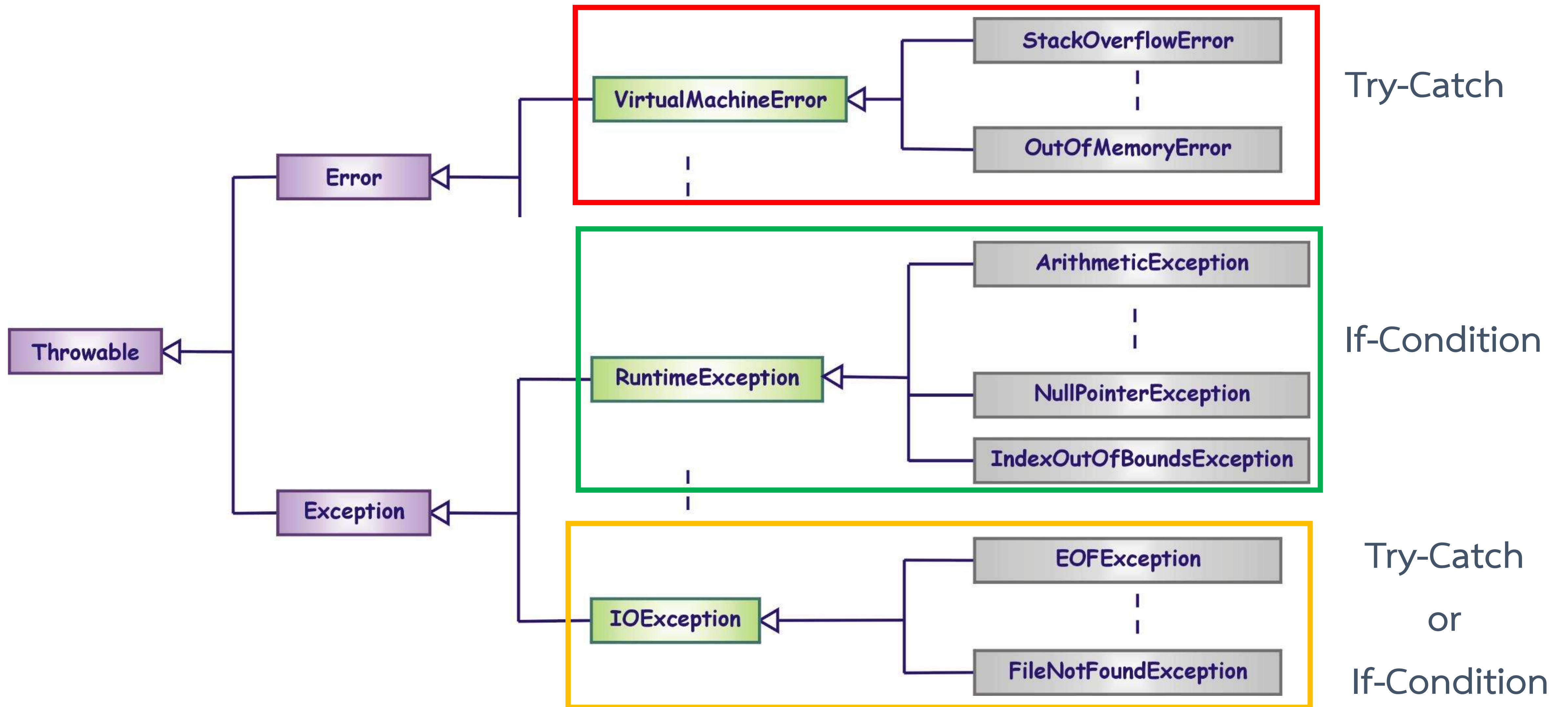
ทำไมต้อง Try-Catch แทนที่จะใช้ If-Condition

โดยทั่วไปแล้ว เราควรจะใช้ Try-Catch จัดการ Exception สำหรับ **ส่วนที่มีการเชื่อมต่อกับปัจจัยภายนอก** โปรแกรม หรือมีการ **เรียกใช้งานทรัพยากรภายนอก** อาทิเช่น อ่าน/เขียนไฟล์, เชื่อมต่อเครื่องแม่ข่าย, เชื่อมต่ออินเทอร์เน็ต, เรียกใช้ API ภายนอก

อย่างไรก็ตาม นักศึกษาต้องพึงพิจารณาไว้ว่าการจัดการ Exception ถือว่าเป็น heavy and expensive operation ทางด้าน ประสิทธิภาพ ถ้า **สามารถเลือกใช้ IF-Condition แทนได้จะทำให้มีประสิทธิภาพที่ดีกว่า** ซึ่งสามารถสรุปได้ดังตารางต่อไปนี้

IF-ELSE	TRY-CATCH
• It checks any condition is true or not, if true then execute the code inside the if block otherwise execute else block. • ‘if-else’ use to handle different conditions using condition checking. • ‘if-else’ is less time consuming than ‘try-catch’. • ‘if-else’ communicate between the data provided to the program and the condition.	• ‘try’ is a section where a code is defined for tested whether the code generates an unexpected result while executing, if any unexpected result found catch block is executed to handle that situation. • In case of ‘try-catch’, the system will check the system generated errors or exception during a executing process or a task. • ‘try-catch’ is more time consuming than ‘if-else’. • ‘try-catch’ communicate between the data with the conditions and the system.

ทำไมต้อง Try-Catch แทนที่จะใช้ If-Condition



เปรียบเทียบการใช้ Try-Catch กับการเขียน If-Condition

```
public class Error03{
    public Chanel channel; // สมมติ
    public void Close()
    {
        if (this.channel.State.equals (CommunicationState.Opened) )
        {
            try {
                this.channel.Close() ;
            } catch (CommunicationException e) {

                // Do Something
            } catch (TimeoutException e) {
                // Do Something
            }
        }
    }
}
```


สรุปเนื้อหาของบท

- ข้อดีประเภทหนึ่งของภาษาจาวา คือ เราสามารถเขียนโปรแกรมให้มีการตรวจจับและจัดการกับข้อผิดพลาดที่อาจเกิดขึ้นได้ โดยที่การทำงานไม่ต้องหยุดลง
- Error เป็นข้อผิดพลาดที่ไม่สามารถแก้ไขและจัดการได้
- Exception เป็นข้อผิดพลาดที่สามารถแก้ไขหรือจัดการได้
- คำสั่ง try และ catch เป็นคำสั่งที่ใช้ในการตรวจจับและจัดการกับข้อผิดพลาดที่อาจเกิดขึ้นได้โดยบล็อกคำสั่ง catch สามารถมีได้มากกว่าหนึ่งบล็อกสำหรับในแต่ละบล็อกคำสั่ง try
- คำสั่ง finally เป็นคำสั่งที่อยู่ต่อจากคำสั่ง try/catch จะถูกเรียกใช้เสมอ ยกเว้นเจอคำสั่ง `System.exit(0);` ก่อนเท่านั้น

สรุปเนื้อหาของบท



- ▶ คำสั่ง throws จะใส่ไว้ตรงคำสั่งประกาศเมธอด สำหรับเมธอดที่ยังไม่ต้องการจัดการกับข้อผิดพลาด แต่จะให้เมธอดที่เรียกใช้เมธอดนี้เป็นตัวจัดการแทน
- ▶ เราสามารถสร้างคลาสประเภท Exception ชนิดใหม่ขึ้นได้ โดยจะต้องสืบทอดมาจากคลาสที่ชื่อ Exception และต้องมีการเรียกใช้ Constructor ของคลาส Exception ด้วย